PLAN — Energy-Ranked Resolution + Contracts as On-Chain EBM Weights

PLAN — Energy-Ranked Resolution + Contracts as On-Chain EBM Weights

Goal (user's words): store contracts on-chain where they *also act as EBM weights*; share data in the ledger to be space/cost efficient; store **primarily on-chain**, local machine as a **frecency cache**. Plus: make the EBM actually **rank which contract fires next** (energy-ranked resolution).

> ***Status (this arc): the value/economics half of this plan SHIPPED.*** *The energy router, CAS dedup, and the* ***reward-not-slash cooperative commons*** *(usage -> USDC to used + verified maintainers; forfeit->honest; nothing burned) are built + proven on* `arc-lean` *as ~18 zero-dependency modules. Canonical spec:* `ECONOMICS-cooperative-commons.md`*. It coexists with (does not supersede) the canonical Vine Doctrine; see DOCTRINE.md ->* `## The Cooperative Commons (arc-lean)`*.*

Corrected premise (verified this session — the earlier draft was wrong). The substrate already trains a real fractal EBM *on-fire* — but on `main`, not arc-lean, and via `ebm.zig`/`ebm_store.zig`/`jepa.zig` (the mislabeled dead `energy_route.zig`/`binary_skill.zig` have since been deleted from arc-lean). What's actually there:

- `ebm.zig`: `E(x) = −cosine(x, centroid)`; the "weight" is a 256-d centroid `mean` + reward-mass `w`. Trained by a reward-weighted Hebbian fold on every declare/satisfy/die, with a 30-day freshness decay. Honest and shallow.
- `jepa.zig`: `E(x,y)=||W·x−y||²`, `W` a 256×256 matrix, real online SGD.
- **Fractal-atomic**: the *same* centroid atom is folded across a self-similar key fan per event — `invoke:{event}`, `:L{layer}`, `:dim:{w6h}`, `:verb:`, `:ptype:`, `:root`. Fractal at the indexing level; one flat atom per key.
- **Ledger-reconstructible**: `replayFromLedger` rebuilds all centroids purely from terminal verdicts (`satisfied->verdict:ok`, `died->verdict:retryable`). So the resolved ledger *is* the weights. Weights are ~1 KB per centroid key.

Two things are missing, and they are this plan:

- **The energy never drives the fire decision.** `route`/`rankByEnergy`/`plan` (argmin-energy selection) exist and are unit-tested, but the live resolver only uses energy to *annotate* the next lever ("resolved-like") and to *gate* tool outputs (typicality). The wire from energy-ranking -> "what fires next" is absent.
- **The weights live in the local ledger, not on-chain**, and are not cost-shared.

Why energy-ranking (question the requirement — the value, and its limits)

Value (where firing is expensive — i.e. on-chain / LLM):

- **Best-first resolution** — fire the lowest-energy (most resolved-like) frontier contract first -> settle in fewer cycles, fewer wasted fires.
- **Cost-aware spend** — energy says *where the SOL/LLM budget pays off* (`payment_energy`/`contract_budget` already score cheapest-chain-that-settles). This is the direct link to the cost goal.
- **Waste/anomaly pruning** — skip candidates whose energy says "nothing that satisfied ever looked like this"; don't spend on dead paths.
- **Self-improving, zero external training** — trains on-fire from the ledger.

Honest limits (state them or it becomes an over-claim):

- Shallow model (cosine-centroid + linear W) = a **soft prior, not a planner**.

- Must never **starve** a contract: energy *prioritizes*, never *forbids*; keep an exploration floor; HOLD stays non-terminal.
- Value is concentrated where firing is **expensive** — negligible for cheap local resolution. So: **prove the router helps (measured) before paying to store weights on-chain.**

Where this lives (a Phase-0 decision)

`main` has the EBM organ (wired, ledger-reconstructible). arc-lean deleted it in the lean. Decide: build on `main` (organ present) vs. yoink the 3 EBM files onto arc-lean first. Do NOT assume — the branches diverged ~17k lines; pick with eyes open.

Risk register (flag before building)

- **Shallow model** — set expectations: better-than-random routing, not reasoning.
- **On-chain cost may be prohibitive** even deduped — Phase 0 measures; fork to commitment-on-chain + weights-off-chain (Arweave) if the numbers say so.
- **Delta-compressibility unproven** — base+delta only saves money if contracts share a base. Verify on real weights before betting on it.
- **Starvation/feedback loops** — a learned router can self-reinforce a bad prior. The fairness floor + exploration is not optional; it's a correctness invariant, mutation-tested.

Phases — prove value BEFORE paying for storage

Phase 0 — Measure, define, decide (no storage code)

- Confirm the energy definition (it exists — `ebm.zig`/`jepa.zig`); decide which fire decision it should drive (frontier order? candidate/verb selection? both?).
- **Baseline the resolver**: instrument current settle-time, fires-to-settle, and (where applicable) SOL/LLM spend on a fixed task set. This is what Phase 1 must beat.
- **Measure Solana rent per KB** (devnet + mainnet); size a real centroid (1 KB) and a real delta; build the cost model; decide on-chain/off-chain boundary.
- **Decide where** (main vs yoink to arc-lean).
- **Gate:** written energy->decision spec, a measured resolver baseline, a cost model with real numbers, a branch decision.

Phase 1 — Energy-ranking router (the missing wire) — LOCAL, prove value

- Wire `ebm_store.route` / `jepa` energy into the resolver's **choice of what fires next**: score the unresolved frontier (and, where a contract has alternative actions/verbs, the candidates) by energy; fire best-first.
- **Fairness floor (correctness invariant):** energy only *reorders*; every live contract still fires within a bounded number of cycles (no starvation); an exploration fraction fires off-prior to keep learning; HOLD stays non-terminal.
- **Gate (this is where "does it help" is PROVEN, not assumed):** on the Phase-0 task set, energy-ranked resolution beats scan-order on ≥1 real metric (fires-to-settle ↓ / settle-time ↓ / wasted-spend ↓) by a stated margin, with the fairness floor mutation-tested (disable it -> a contract starves -> RED). If it does NOT beat baseline, STOP — the router isn't worth wiring, and no on-chain storage follows.

Phase 2 — Content-addressed on-chain weight store + free dedup

(only if Phase 1 proved value AND Phase 0 cost is feasible)

- Content-addressed chunks: `blob=(sha256(bytes),bytes)`; **PDA = derive(hash)** -> identical content = same account = write-if-absent = **dedup for free** (the "share data" cost win falls out of content-addressing).

- Store/read a chunk on-chain (extend `iq_ledger`, or IQLabs `codeIn`-style chunking / adopt that brick). Local CAS cache keyed by hash.
- **Gate:** store a blob, store it again = 0 new on-chain bytes; evict local, read back from chain = identical; cost-per-unique-KB measured; corrupt a chunk -> hash-mismatch caught.

Phase 3 — Base + delta EBM (cost lever)

- Shared base centroids stored once; per-contract weights = base_ref + small delta. Contract row carries `{lineage, pointers, base_ref, delta_ref, commit}`.
- **Gate:** reconstructed weights == full; two contracts sharing a base pay only their deltas (measured); inference/routing output correct; wrong delta -> RED. Also *verifies* the delta-compressibility assumption (risk #3).

Phase 4 — Frecency cache (chain canonical, local hot)

- Local cache with frecency eviction (frequency × recency decay), sidecar index; miss -> fetch chain; hit -> local. Tiers: hot=full local, warm=commit+fetch, cold=commit-only.
- **Gate:** `rm -rf cache && refetch -> identical`; hot survives, cold evicts, size bounded; measured hit-rate on a real access trace; cold miss + no network -> HOLD (never fabricate).

Phase 5 — Compression + quantization (measured byte savings)

- gzip/zstd + int8/int4 shards, stacked with dedup+delta.
- **Gate:** measured on-chain-bytes reduction vs Phase-2 baseline; correctness within a stated tolerance; routing output unchanged beyond tolerance.

Phase 6 — Confidentiality (optional; wires the earlier discussion)

- Encrypt chunks (`payload_crypto` envelope / IQLabs `multiEncrypt`) -> on-chain ciphertext; receptor grant gates key release; PRE for dynamic grants; ZK only for property/anonymity — not for storage.
- **Gate:** unauthorized can't decrypt; authorized can; dedup/cost preserved.

Cross-cutting invariants

- **Prove value before paying for storage** (Phase 1 gate is the fork).
- **Fairness floor:** energy prioritizes, never starves; exploration keeps learning; HOLD non-terminal. Mutation-tested.
- **Chain canonical, local reconstructable:** every phase survives `rm -rf cache && refetch -> identical`.
- **Agency spend-gate first** (`agency.zig`): every on-chain write checks the SOL budget before signing, or a routing/store loop drains the wallet.
- **Everything mutation-proven + measured.** No un-measured estimate ships; no test that can't fail. Honest naming (say "centroid"/"linear"/"encrypted", not "zk").

Phase 0 RESULTS (measured 2026-07-15, mainnet RPC + deterministic formula agree)

Rent-exempt minimum (a recoverable DEPOSIT, not a burn; per-write fee ~0.000005 SOL):

data size	SOL locked	~USD @ $150
32 B (a commitment hash)	0.00100	$0.15
1 KB (a centroid)	0.00802	$1.20
4 KB	0.0294	$4.41
8 KB	0.0579	$8.69

data size	SOL locked	~USD @ $150
256 KB (JEPA matrix)	~1.83	~$274

Boundary decision (grounded in these numbers):

- **On-chain (store the bytes):** small contract rows + shared centroids (~1 KB) + per-contract weight *deltas* — anything ≤ **~4 KB** (≤ ~0.03 SOL locked). Free dedup via content-addressed PDA means a shared base/centroid is paid **once**.
- **Off-chain (commitment only):** any tensor **> ~4 KB** (the JEPA 256 KB matrix, real NN weights MB–GB) goes to Arweave/IPFS/local; only its **32-byte commitment** (~0.001 SOL) is anchored on-chain. On-chain storage of large tensors is infeasible and unnecessary.
- **Frecency bounds locked capital to the working set:** hot contracts fully on-chain; cold = commitment-only, rehydrated on use. Locked SOL scales with the *hot set*, not total history.

Economics honest take: the burn is negligible; the number that matters is *locked capital*. For 10 K contracts at ~300 B on-chain each (base+delta+dedup): ~30 SOL locked (~$4.5 K, recoverable) + ~$8 in fees. Feasible and dedup/delta/ frecency are exactly the levers that shrink it. "Contracts as EBM weights on-chain" is viable **because the weights are ~1 KB centroids, not big tensors** — the big tensors stay off-chain by commitment.

Where: build on arc-lean; yoink main's self-contained 9-file Solana engine (analysis: zero calls into main's core; rests on `id`/`keystore` arc-lean already has). Port the `agency.zig` spend-gate FIRST.

Build spine

`0 measure/define` -> `1 energy router (prove value)` -> *fork: worth it?* -> `2 CAS+dedup` -> `3 base+delta` -> `4 frecency cache` -> `5 compress/quant` -> `6 confidentiality`. Phase 1 is the load-bearing proof; 2–6 only follow if it pays.

Status — Phase 1 (energy router): BUILT + benchmarked (2026-07)

The energy router is live: `contract recommend "<goal>"` (also `contract_recommend` C-ABI, and aiko-cli via FFI) — ONE verb that returns both the argmin-energy binary and the nearest PROVEN command for it (the self-documented usage exemplar). It trains a per-binary prototype PURELY from the ledger (satisfied goal -> +1 toward the binary its action named, died -> −1) — no hardcoded lexicon, no cold-start seed: a binary becomes routable only after real use. aiko-cli consults it as a soft prior each turn and feeds it every successful command (full argv, not just the binary), so selection AND usage both learn from real work. No LLM.

Embedding: the shallow feature-hash capped accuracy (44% -> 57% after IDF-approx stopwords; grep 11% — pure synonymy was unreachable). Fixed by baking a REAL learned sentence embedder: bge-small-en-v1.5 as a pure-Zig 12-layer BERT over per-row int8 weights (`bge.zig`, @embedFile'd, zero runtime deps — matches its numpy golden reference at cosine > 0.999).

Honest benchmark (`zig test libcontract/src/route_bench.zig -OReleaseFast`, 54 goals × 6 binaries, leave-one-out, pure ledger): **88.9%** — 5.3× random (16.7%), keyword 1.9%; grep 89%, curl 100%; ~85% from a single training example per binary (k=1). Measured against real alternatives on the identical harness: GloVe-100 66.7%, nomic-embed 87.0%, bge-small 88.9%, Qwen3-0.6B 96.3% (the ceiling; 20× the size for +7pts — not worth the bundle). Phase-1 fork verdict: a genuinely trustworthy soft prior (~9/10) that generalizes by meaning and self-documents how each tool was successfully used.